

Práctica 4 - Hilos y comunicación MQTT

Alejandro Otero Vázquez y Mario Bravo López

Fecha de entrega: 13 de diciembre

En este documento se recogen aquellas secciones o explicaciones que hemos considerado más importantes recalcar para poder explicar con claridad la práctica 4 así como el código que se ha usado en la misma.

1 Sensor de distancia

Para poder realizar esta práctica se usarán dos sensores, el ya conocido LSM6DS33 para sensar la temperatura y el sensor GP2Y0A41SK0F, sensor cuya funcionalidad recae en medir distancias. Este sensor electro-óptico está compuesto por un PSD (position sensitive detector), un IR-LED (infrared emitting diode) y un circuito para procesar la señal. Evidentemente la salida del sensor será de tensión y estará relacionada con la distancia de la siguiente manera:

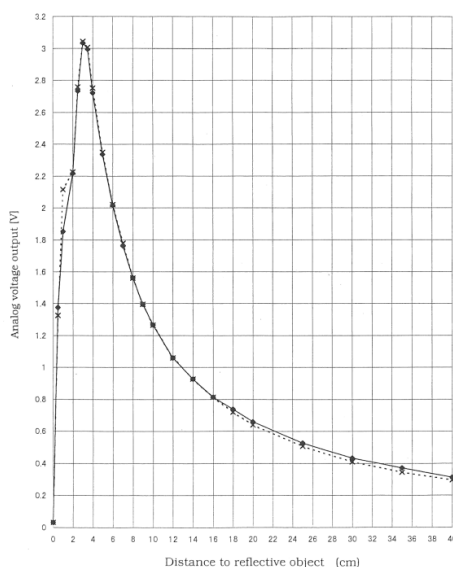


Figure 1: Relación distancia voltaje sensor GP2Y0A41SK0F

Esta relación entre la tensión de la salida y la distancia tiene un problema la multievaluación, pues como se observa en la gráfica adjunta, para distancias diferentes tenemos asociados tensiones iguales. La solución a este problema será considerar que todas las tensiones que se midan serán para distancias a la derecha del máximo presente en la gráfica y consecuentemente como se puede observar en la gráfica las distancias a medir tienen que ser superiores a 3 cm para seguir el criterio que hemos comentado.

Por último nos gustaría recordar que la salida de este sensor será una tensión por lo tanto deberemos hacer uso del ADC de la Raspberry Pi, además de realizar la configuración de la comunicación SPI usando *mcp3004Setup (pinbase, spi_channel)*. Como esta funcionalidad ya se expresó en la segunda práctica no nos detendremos a explicar que pasos hay que seguir, simplemente iremos al grano siguiendo lo aprendido en dicha práctica.

1.1 Calibración

Hemos optado por calibrar el sistema directamente a partir de la gráfica que brinda el datasheet. Obteniendo la relación lineal que existe entre los píxeles de la imagen con los valores para distancia y voltaje somos capaces de muestrear la función con intervalos muy bajos. Al haber tan poco espacio entre valores consecutivos puede aproximarse lineal la función en cada intervalo y ponderar de este modo cada valor analógico medido por el sensor.

Para obtener la relación entre la imagen y los valores de las variables se ha utilizado esta web:

<https://apps.automeris.io/wpd/> siguiendo las indicaciones del siguiente vídeo de youtube:

https://www.youtube.com/watch?v=jXTgLhKLhSo&ab_channel=Latex%26Matlab. El estudio da como resultado un archivo csv que contiene 70 muestras de valores de distancia y voltaje obtenidos a partir de la gráfica.

2 Sensor de temperatura

Como este sensor se ha usado con anterioridad para la práctica 3, no se le dará especial importancia en este documento. Únicamente deberemos recordar la comunicación se hacía mediante el protocolo I2C, se ha de acceder a su datasheet para ver que registros se debían modificar para poder activar la lectura de temperatura o gravedad por parte del sensor. Y posteriormente usar los registros dedicados para medir la temperatura y concatenarlos posteriormente como ya se había explicado con anterioridad.

3 Funciones

En el código además de la creación de hilos y la comunicación con MQTT que explicaremos más adelante se usan dos funciones para medir con los dos sensores que ya hemos descrito previamente, estas funciones son: **get_temperature** y **get_distance**, ambas funciones son de tipo *void ** para poder implementarlas posteriormente en los hilos como veremos. Otra característica de las dos es que ambas tienen como parámetro una variable puntero a la estructura mosquito.

Empezaremos analizando la más sencilla de las dos **get_temperature**: En esta primera función se hace un bucle infinito usando *while(1)*, en el cual se llevará a cabo la lectura típica de los registros asociados a la temperatura del LSM6DS33, usando la librería de wiringPi para la comunicación I2C y su posterior concatenación y procesado. El paso innovador que se introduce en esta función respecto a la de prácticas anteriores es la línea:

```
1    publish_sensor_temp((struct mosquito *) mosq, temp_data);
```

La cual toma como entrada el identificador para la estructura mosquito y el dato de la temperatura y funciona según:

```
1    void publish_sensor_temp(struct mosquito *mosq, float temp)
2    {
3        char payload[20];
4        int rc;
5        snprintf(payload, sizeof(payload), "%lf", temp);
6        rc = mosquito_publish(mosq, NULL, "RaspberryPi/temperature", strlen
          (payload), payload, 2, false);
7        if(rc != MOSQ_ERR_SUCCESS){
8            fprintf(stderr, "Error publishing: %s\n", mosquito_strerror(rc)
          );
9        }
10 }
```

En esta función en primera instancia se crea la cadena de caracteres payload donde se almacenará usando **snprintf()** la información de la temperatura que es un variable de tipo flotante, para facilitar su lectura. Por

último usamos **mosquitto_publish()**, para publicar dicha cadena de caracteres en el tópico "Raspberry/temperature", para usar esta función tenemos que usar el puntero a la estructura mosquito *mosq* además de incluir la longitud de nuestra cadena de caracteres en bytes.

La segunda función que se usa recursivamente en el código es **get_distance** que sigue la siguiente estructura:

```
1 void *get_distance(void *mosq){
2
3 FILE *file;
4
5 float x[MAX_ROWS+1];
6 float voltage[MAX_ROWS+1];
7 int fila = 0;
8
9 file = fopen("calibracion.csv","r");
10
11 if (file == NULL) {
12     printf("No se pudo abrir el archivo\n");
13     exit(1);
14 }
15
16 // Leer el archivo CSV
17 while (fila < MAX_ROWS) {
18     fscanf(file,"%f;%f",&x[fila],&voltage[fila]);
19     fila++;
20 }
21
22 fclose(file);
23
24 ///// Calculo de periodo de muestreo
25 float t_muestreo=1e3/(3); //Pasamos el tiempo de muestreo a
    microsegundos para compararlo con los de medida del ADC
26 float measure[2];
27 float alpha = 0.9;
28 float t1,t2;
29 float dist;
30
31 while(1) {
32     t1 = millis();
33
34     measure[0] = measure[1];
35     measure[1] = analogRead(pinbase)*3.3/1023; //Paso a voltios
36     measure[1] = alpha*measure[1]+(1-alpha)*measure[0];
37
38     dist = getDistance(measure[1],&x[0],&voltage[0]);
39
40     //printf("distancia = %f cm \n",dist);
41     publish_sensor_distance((struct mosquito *) mosq,dist);
42     t2 = millis();
43     if(t_muestreo>(t2-t1))
44         delay(t_muestreo-(t2-t1)); //EL tiempo entre medida y medida menos
        lo que tardamos en leer el pin, el retardo.
45
46 }
```

```

47 return (void *) NULL;
48 }

```

En las primeras líneas del código nos dedicamos a la creación de un archivo `*file` y cargar en el nuestro fichero de excel con la calibración, para posteriormente usar un bucle para cargar los datos de dicho archivo a las variables x y *voltage*, usando `fscanf()`. Posteriormente entramos en un bucle infinito en el cual se realiza la medida del sensor de distancia usando la función `analogRead()` de la librería `wiringPi`, hay que destacar que estas medidas se pasarán por un filtro pasa bajo.

Sin embargo estas medidas son de *voltage* y deseamos pasarlas a distancia, para ello usaremos la función `getDistance()`:

```

1   float getDistance(float measure, float *x, float *voltage){
2   float dist;
3
4   int n = 0;
5
6   while(n<MAX_ROWS){
7       if(measure<*voltage){
8           x++;
9           voltage++;
10      }
11      n++;
12  }
13  float interval = *voltage-*(voltage-1);
14  float frac = (measure-*(voltage-1))/interval;
15  //printf("%f\n",interval);
16  dist = *(x-1) + frac*(x-*(x-1));
17  return dist;
18 }

```

Que de manera resumida, compara el valor de *meassure*, que sera la medida del sensor, con los valores guardados en la variable `voltage[]` iterativamente hasta que uno de los valores del vector `voltage[]` es menor que *meassure*. Dicho valor de tensión tiene una distancia asociada `x[]`, sin embargo esta distancia no es una buena aproximación para el valor real que nosotros tenemos. Es por ello que se calcula una variable *frac* que representa la diferencia relativa de la medida al valor de tensión más cercano por abajo. Conocida esta fracción se usa para calcular la distancia usando los valores del vector `x[]` y el valor obtenido para *frac* para aproximar el valor real que se tiene de la distancia.

Ya por último obtenida la variable *dist* procedemos a publicarla usando la función `publish_sensor_distance()` que sigue una estructura similar a la función usada para publicar la temperatura, pero publicando el mensaje en el tópico "RaspberryPi/distance".

4 Implementación con Hilos

Esta práctica esta pensada para realizar procesos en paralelo, haciendo uso de los conocidos hilos, mediante la biblioteca `libpthread`, con la cual implementaremos distintos hilos para realizar las funciones de medir temperatura y distancia de manera simultánea.

En el código adjuntado se usan dos hilos:

```

1 pthread_t thread1,thread2; //Aquí definimos los identificadores de los
    dos hilos

```

Estos dos hilos tendrán dos funciones distintas. Como ya se ha el primer hilo se usará para la medida de temperatura y el segundo para la medida de distancia, además en el código se confirma que se han creado

los hilos:

```
1         if ((rc=pthread_create(&thread1, NULL, &get_temperature, (void *)
           mosq))) {
2     printf("Fallo de pthread:\n");}
3         if ((rc=pthread_create(&thread2, NULL, &get_distance, (void *) mosq)
           )) {
4     printf("Fallo de pthread:\n");}
```

Al primer hilo se le asocia la función para medir la temperatura mediante la función **get_temperature** y se le pasará como argumento el identificador de la estructura a puntero *mosq*. Para el segundo hilo de igual manera se le asociará la función **get_distance**. Posteriormente se pondrán ambos hilos a espera usando *pthread_join()*

5 Conexión MQTT

En esta práctica una de los principales objetivos es explotar el concepto de la conexión MQTT, basado en la relación subscriptor/publicador, en la cual será necesario la presencia de un Broker para direccionar y poner orden en estas conexiones, en esta práctica como se nos ha indicado se ha usado el broker proporcionado por "test.mosquitto.org", en conctreo se usará el puerto 1883.

Hay que destacar que antes de usar las funciones de publicación: **publish_sensor_distance()** y **publish_sensor_temperature()** para poder publicar, primero hemos de crear una **struct_mosquitto *mosq**, posteriormente se genera el cliente con *mosquitto_new* y se establece la conexión con el broker mediante *mosquitto_connect()* hay que destacar que ambas funcionalidades están definidas en la librería de mosquitto y toman como una de las variables de entrada la variable puntero *mosq* a la estructura mosquitto.

Otras consideraciones a la hora de usar mosquitto es la configuraciones de los callbacks haciendo uso de las funciones *mosquitto_connect_callback_set* y *mosquitto_publish_callback_set*.

5.1 RaspberryPi/Distance

En esta sección comentaremos las diferencias del código "suscribe_dist.c" para subscribirnos al broker en el tópico correspondiente a la distancia. En la función main el comportamiento distinto respecto a la función publicadora recae en los callbacks en este caso se usaran: *mosquitto_subscribe_callback_set* y *mosquitto_message_callback_set*. En especial nos gustaría hacer especial inca pie en la función **on_message**:

```
1     void on_message(struct mosquitto *mosq, void *obj, const struct
           mosquitto_message *msg)
2     {
3         /* This blindly prints the payload, but the payload can be anything
           so take care. */
4         printf("%s %d dist: %s cm\n", msg->topic, msg->qos, (char *)msg->
           payload);
5         distance = atof((char *)msg->payload);
6         //printf("%f \n", temperature);
7
8         if(distance<10){
9             digitalWrite(LED1,1);
10        }
11
12    }
```

En la cual, en el caso de que la distancia que se este obteniendo al subscribirse al tópico sea menor de 10 cm se encienda un led para avisarnos de la cercanía del sensor a un objeto.

5.2 RaspberryPi/Temperature

El tercer código de este trabajo "suscribe_temp.c" se encarga de subscribirse al tópico de la temperatura, el código para realizar esta función es muy similar al comentado anteriormente en el caso de la distancia con la diferencia que a la hora de definir los callbacks la función para mostrar los mensajes **on_message** tiene una variación:

```
1     void on_message(struct mosquitto *mosq, void *obj, const struct
      mosquitto_message *msg)
2 {
3     /* This blindly prints the payload, but the payload can be anything
      so take care. */
4     printf("%s %d T: %s C\n", msg->topic, msg->qos, (char *)msg->payload
      );
5     temperature = atof((char *)msg->payload);
6     //printf("%f \n",temperature);
7     if(n==0){
8         tref = temperature;
9         n++;
10    }
11    if(temperature-tref>1){
12        digitalWrite(LED1,1);
13        digitalWrite(LED2,0);
14    }
15    if(temperature-tref<-1){
16        digitalWrite(LED1,0);
17        digitalWrite(LED2,1);
18    }
19
20 }
```

De esta función es destacable que en la primera iteración se define una temperatura de referencia, la cual si varía más de un grado tanto para arriba como para abajo, se nos provocarán distintas configuraciones de los LED's presentes en la Raspberry Pi.